

DOCUMENTAÇÃO TÉCNICA

Documentação de Testes Automatizados (BDD e API)

Projeto: Cidades ESG Inteligentes (Waste Reporting API)

1. Introdução e Contexto ESG

Visando promover a sustentabilidade, responsabilidade social e governança (**ESG**), este projeto implementa uma suíte de Testes Automatizados focada em validar as principais regras de negócios e a integridade da **API RESTful de Gestão de Resíduos**.

A estratégia adotada foi o **Behavior Driven Development (BDD)**, utilizando o framework **SpecFlow (Gherkin)**. Essa abordagem garante que os testes não sejam apenas técnicos, mas sirvam como documentação viva, atestando que funcionalidades críticas de Rastreabilidade e Governança Ambiental estejam funcionando conforme o esperado.

2. Configurações de Infraestrutura de Testes

Para garantir a isolamento, repetibilidade e eficiência dos testes (aderindo às melhores práticas de **CI/CD**), as seguintes configurações foram aplicadas:

1. **Banco de Dados In-Memory:** Quando a variável `ASPNETCORE_ENVIRONMENT` é definida como `Testing`, a API sobe utilizando o `Microsoft.EntityFrameworkCore.InMemory`. Isso permite testar todos os endpoints sem sujar o banco de produção (Oracle) e acelera drasticamente o tempo de feedback.
2. **Setup Automatizado (Admin Seed):** Usuários de teste (como o Cidadão e o Administrador) são previamente semeados no banco em memória, garantindo a validação da Governança (bloqueios via Role).

3. **Orquestração via Docker Compose:** Foi criado um arquivo específico `docker-compose.test.yml`. Ele sobe um container isolado contendo o ambiente `.NET SDK 9.0` que aguarda a API ficar disponível, roda o comando `dotnet test` e desliga (`abort-on-container-exit`) atestando sucesso (`exit code 0`).
4. **Integração Contínua (GitHub Actions):** O fluxo de trabalho `ci.yml` foi adaptado para executar as baterias de testes em nuvem, rejeitando a branch caso qualquer cenário BDD quebre.

```
docker-compose.test.yml
> Run All Services
1 services:
  > Run Service
2 api:
3   image: wastereporting-api-test
4   build:
5     context: .
6     dockerfile: src/WasteReporting.API/Dockerfile
7   ports:
8     - '5001:8080'
9   environment:
10    - ASPNETCORE_ENVIRONMENT=Testing
11    - ConnectionStrings__DatabaseConnection=${DATABASE_CONNECTION:-}
12    - JwtSettings__SecretKey=${JWT_SECRET_KEY:-SuperSecretKey123!ForTestingOnly}
13    - JwtSettings__Issuer=test
14    - JwtSettings__Audience=test
15    - JwtSettings__ExpirationMinutes=60
16
  > Run Service
17 bdd-tests:
18   image: mcr.microsoft.com/dotnet/sdk:9.0
19   depends_on:
20     - api
21   volumes:
22     - ./app
23   working_dir: /app/src/WasteReporting.BDDTests
24   # Aguarda a API subir e executa os testes
25   command: >
26     bash -c "sleep 15 && dotnet test --verbosity normal"
27   environment:
28     - API_URL=http://api:8080
29
```

Configuração do Orquestrador de Testes BDD

```
if (builder.Environment.IsEnvironment("Testing"))
{
    builder.Services.AddDbContext<AppDbContext>(options =>
    {
        options.UseInMemoryDatabase("WasteReportingTestDB");
    });
}
else
{
    builder.Services.AddDbContext<AppDbContext>(options =>
    {
        options.UseOracle(builder.Configuration.GetConnectionString("DatabaseConnection"), b => b.UseOracleSQLCompatibility(
        OracleSQLCompatibility.DatabaseVersion19));
    });
}
```

Banco de Dados Em Memória.

3. Detalhamento dos Cenários BDD (Gherkin)

A cobertura de testes abrange todos os 7 *controllers* críticos da aplicação, divididos em 4 blocos de funcionalidades (**Features**).

3.1. Autenticação e Compliance (**Auth.feature**)

Garante que a API seja fechada contra acessos indevidos, validando a geração correta de tokens **JWT**.

- **Cenário 1 (Login Válido):** Envia e-mail e senha corretos. O status esperado é **200 OK** e o payload JSON de retorno é confrontado contra o contrato rigoroso **AuthResponseSchema.json**.
- **Cenário 2 (Login Inválido):** Envia credenciais erradas e assegura que a API retorne corretamente o status de bloqueio **401 Unauthorized**.

3.2. Relatos e Auditoria Cidadã (**Reports.feature**)

Valida os fluxos de denúncia de lixo irregular por cidadãos e a respectiva gestão pelos administradores governamentais.

- **Cenário 1 (Criar Relato):** Um cidadão autenticado envia um POST para criar um descarte irregular (**201 Created**). Testa-se se o contrato retornado atende ao **ReportResponseSchema.json**.
- **Cenário 2 (Listar Meus Relatos):** Cidadão consome a rota GET para ver apenas os seus relatos (Garantia de Privacidade).
- **Cenários 3 e 4 (Gestão Admin):** O administrador lista todos os relatos do sistema e dispara um PUT validando que consegue mudar o status da denúncia (Ex: para "RESOLVIDO").

3.3. Cadastros Estruturais (**Management.feature**)

Cobre as rotas vitais para a operação logística do município.

- **Cenários de Inserção:** Dispara POSTs validados para a criação de "Pontos de Coleta", "Empresas Recicladoras", "Destinos Finais (Lixões/Aterros)" e "Tipos de Resíduos". Todos esperando **201 Created**.
- **Cenário de Leitura:** Executa um GET para listar os tipos de resíduos e provar o retorno dos dados estruturais de reciclagem (**200 OK**).

3.4. Fluxo e Rastreo de Coletas (**Collections.feature**)

Responsável por atestar o núcleo da operação **ESG**: o vínculo do lixo irregular que foi recolhido ao seu destino sustentável final.

- **Cenários de Agendamento:** Valida as ações de Agendar Coleta (**POST**) e Listar Coletas (**GET**).
- **Cenários de Associação de Resíduos:** O teste cria um POST para atrelar um tipo específico de Lixo (Waste) pesado em KG à uma Coleta e depois um DELETE para desfazer o vínculo. Isso valida o peso da carga recuperada (**Rastreabilidade**).
- **Cenários de Atualização:** Testes de PUT e DELETE finais para atestar que os status da coleta possam fluir de "Agendada" para "Concluída".

3.5. Definição de Contratos de API

- Para garantir a integridade da comunicação e evitar quebras de contrato em futuras versões, foram implementados esquemas de validação. Abaixo, o exemplo do contrato utilizado para a autenticação:

```

1  {
2    "type": "object",
3    "properties": {
4      "token": { "type": "string" },
5      "username": { "type": "string" },
6      "email": { "type": "string" },
7      "role": { "type": "string" }
8    },
9    "required": ["token", "username", "email", "role"]
10 }
11 %L for Agent

```

Arquivo: AuthResponseSchema.json

4. Evidências de Execução dos Testes

4.1. Sucesso na Execução Local

A execução orquestrada rodou de ponta a ponta sem falhas, validando todas as conexões **HTTP** (GET, POST, PUT, DELETE), verificações de código de status e validação profunda de propriedades JSON de contrato.

```

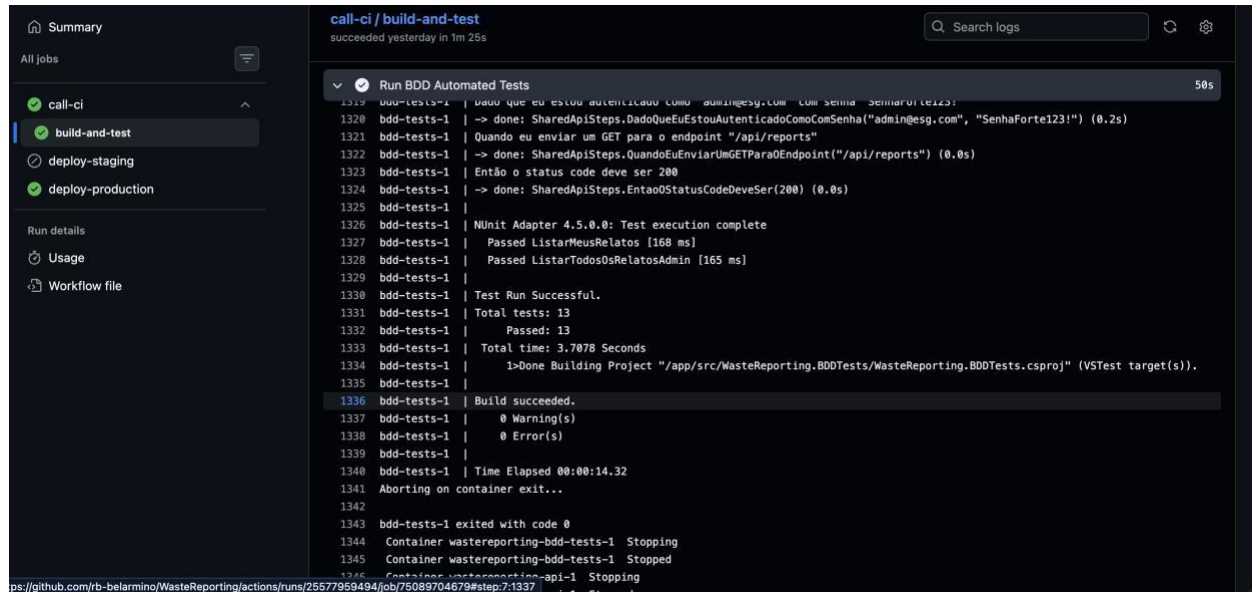
bdd-tests-1 | -> done: SharedApiSteps.DadoQueEuEstouAutenticadoComoComSenha("admin@esg.com"
, "SenhaForte123!") (0.1s)
bdd-tests-1 | Quando eu enviar um GET para o endpoint "/api/reports"
bdd-tests-1 | -> done: SharedApiSteps.QuandoEuEnviarUmGETParaOEndpoint("/api/reports") (0.0
s)
bdd-tests-1 | Então o status code deve ser 200
bdd-tests-1 | -> done: SharedApiSteps.EntaoOStatusCodeDeveSer(200) (0.0s)
bdd-tests-1 |
bdd-tests-1 | NUnit Adapter 4.5.0.0: Test execution complete
bdd-tests-1 | Passed AssociarResiduoAColeta [194 ms]
bdd-tests-1 | Passed AtualizarEDeletarColeta [161 ms]
bdd-tests-1 | Passed AtualizarStatusDoRelatoAdmin [466 ms]
bdd-tests-1 | Passed CriarUmRelatoDeDescarteIrregular [153 ms]
bdd-tests-1 | Passed ListarMeusRelatos [159 ms]
bdd-tests-1 | Passed ListarTodosOsRelatosAdmin [155 ms]
bdd-tests-1 |
bdd-tests-1 | Test Run Successful.
bdd-tests-1 | Total tests: 13
bdd-tests-1 | Passed: 13
bdd-tests-1 | Total time: 3.0189 Seconds
bdd-tests-1 | 1>Done Building Project "/app/src/WasteReporting.BDDTests/WasteReporting
.BDDTests.csproj" (VSTest target(s)).
bdd-tests-1 |
bdd-tests-1 | Build succeeded.
bdd-tests-1 | 0 Warning(s)
bdd-tests-1 | 0 Error(s)
bdd-tests-1 |
bdd-tests-1 | Time Elapsed 00:00:04.01
bdd-tests-1 | exited with code 0

```

Console de execução confirmando a passagem de todos os 13 cenários Gherkin implementados.

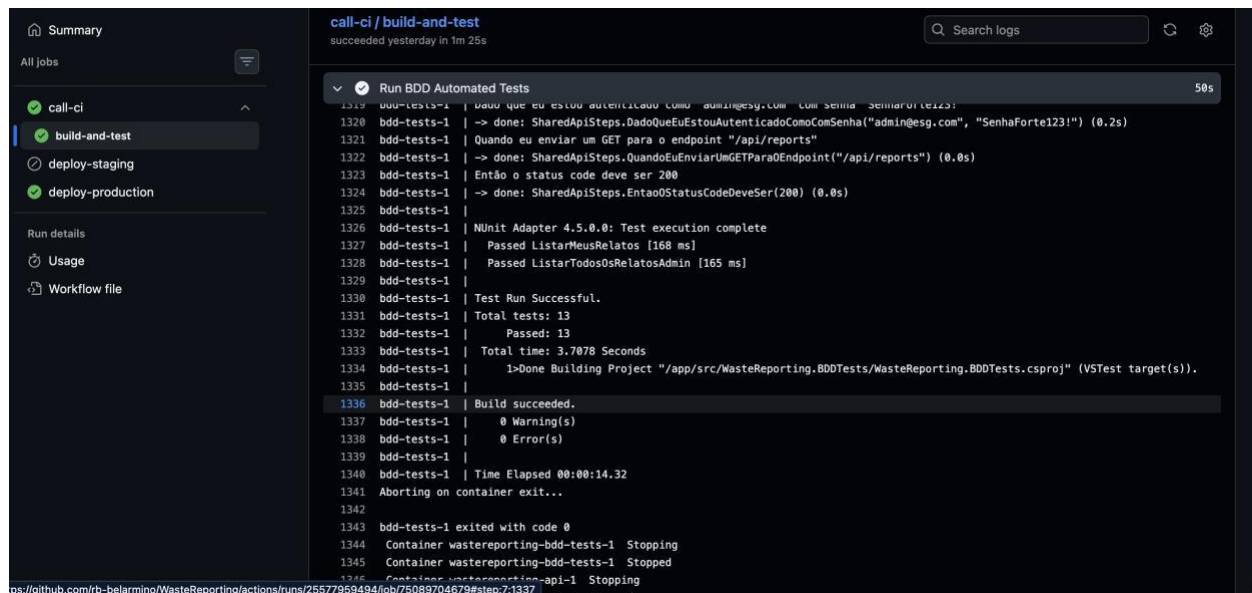
4.2. Sucesso na Esteira CI/CD (GitHub Actions)

Os mesmos testes também rodaram com sucesso no ambiente em nuvem integrado.



The screenshot displays the GitHub Actions interface for a workflow named 'call-ci / build-and-test'. The left sidebar shows a list of jobs: 'call-ci', 'build-and-test' (selected), 'deploy-staging', and 'deploy-production'. The main panel shows the details of the 'Run BDD Automated Tests' step, which has a duration of 50s. The log output shows the following steps:

```
1319 bdd-tests-1 | Passo que eu estou autenticado como admin@esg.com com senha SenhaForte123! (0.2s)
1320 bdd-tests-1 | -> done: SharedApiSteps.DadoQueEuEstouAutenticadoComoComSenha("admin@esg.com", "SenhaForte123!") (0.2s)
1321 bdd-tests-1 | Quando eu enviar um GET para o endpoint "/api/reports"
1322 bdd-tests-1 | -> done: SharedApiSteps.QuandoEuEnviarUmGETParaOEndpoint("/api/reports") (0.0s)
1323 bdd-tests-1 | Então o status code deve ser 200
1324 bdd-tests-1 | -> done: SharedApiSteps.EntaoOStatusCodeDeveSer(200) (0.0s)
1325 bdd-tests-1 |
1326 bdd-tests-1 | NUnit Adapter 4.5.0.0: Test execution complete
1327 bdd-tests-1 | Passed ListarMeusRelatos [168 ms]
1328 bdd-tests-1 | Passed ListarTodosOsRelatosAdmin [165 ms]
1329 bdd-tests-1 |
1330 bdd-tests-1 | Test Run Successful.
1331 bdd-tests-1 | Total tests: 13
1332 bdd-tests-1 | Passed: 13
1333 bdd-tests-1 | Total time: 3.7078 Seconds
1334 bdd-tests-1 | 1>Done Building Project "/app/src/WasteReporting.BDDTests/WasteReporting.BDDTests.csproj" (VSTest target(s)).
1335 bdd-tests-1 |
1336 bdd-tests-1 | Build succeeded.
1337 bdd-tests-1 | 0 Warning(s)
1338 bdd-tests-1 | 0 Error(s)
1339 bdd-tests-1 |
1340 bdd-tests-1 | Time Elapsed 00:00:14.32
1341 Aborting on container exit...
1342
1343 bdd-tests-1 exited with code 0
1344 Container wastereporting-bdd-tests-1 Stopping
1345 Container wastereporting-bdd-tests-1 Stopped
1346 Container wastereporting-api-1 Stopping
```



This is a duplicate of the screenshot above, showing the same GitHub Actions workflow run for 'call-ci / build-and-test' with successful BDD automated tests. The log output is identical to the one in the previous block.

5. Conclusão

A aplicação demonstrou conformidade total em todas as suas rotas, reforçando o pilar de Governança **ESG** e garantindo confiabilidade no fluxo de informações socioambientais do município.